Single-cell RNA-seq, Technology and Quality Control

Yongjin Park

February 3, 2026

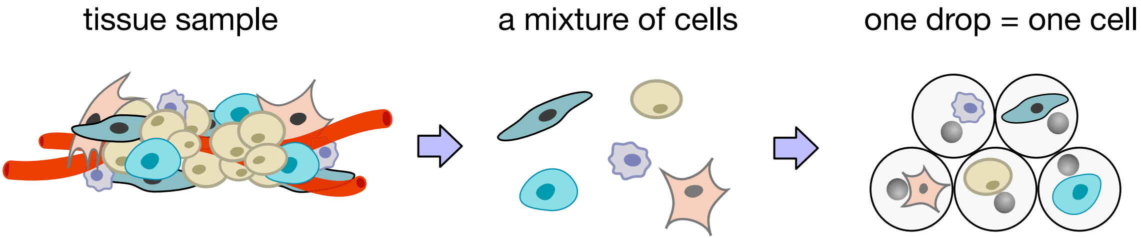
Learning objectives

- Introduction to single-cell RNA-seq technology
- Understand single-cell RNA-seq data structure
- Introduction to analysis workflow
  - Data quality control
  - Exploratory data analysis

Today's lecture

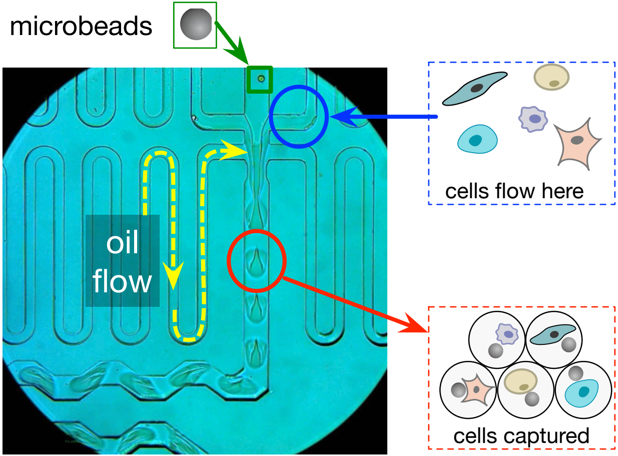
1. Single-cell sequencing technology
2. Basic quality control
3. Data normalization across many batches

Droplet-based single-cell sequencing technology



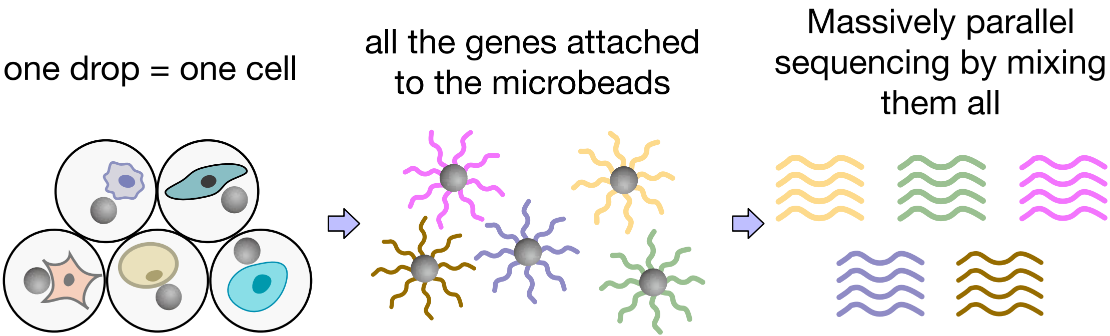
(Macosko et al. 2015)

Idea 1: Capture one cell with a microbead in a droplet



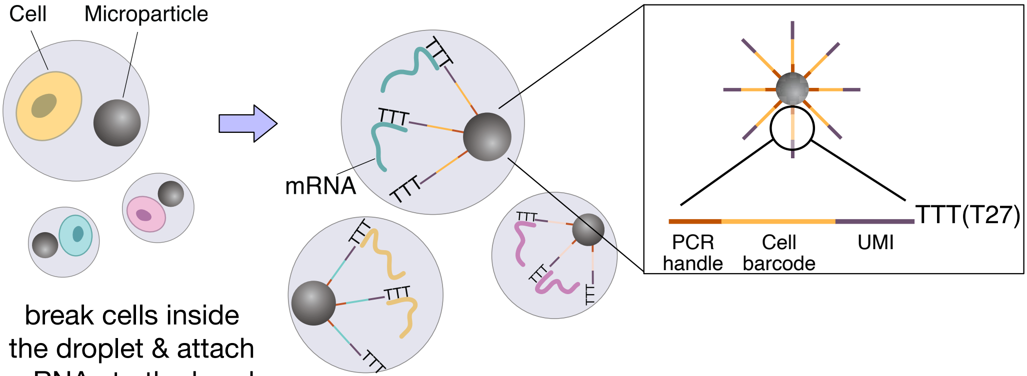
(Macosko et al. 2015)

Idea 2: Massively-parallel sequencing followed by cell-specific barcoding



(Macosko et al. 2015)

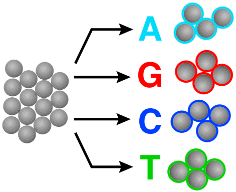
Idea 3: How do we keep track of mRNA short reads' membership to a certain droplet?



break cells inside the droplet & attach mRNAs to the bead

(Macosko et al. 2015)

How do we construct millions of unique barcodes? Use DNA as a hashing function!

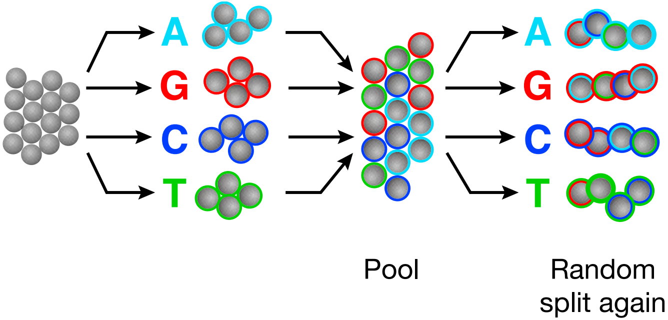


Tag each set with the same base (A,G,C,T)

Randomly split the beads into 4 sets

(Macosko et al. 2015)

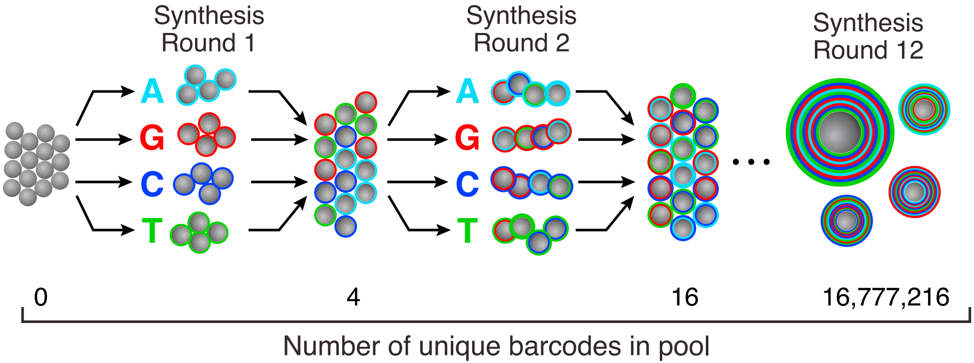
How do we construct millions of unique barcodes? Use DNA as a hashing function!



Tag each set with the same base (A,G,C,T)

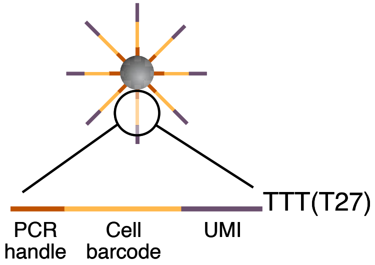
(Macosko et al. 2015)

How do we construct millions of unique barcodes? Use DNA as a hashing function!

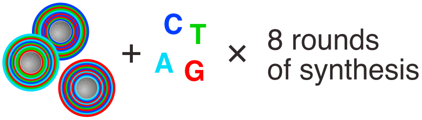


(Macosko et al. 2015)

The lengths of barcode sequences determine data dimensionality



4^{12} 4^{8}

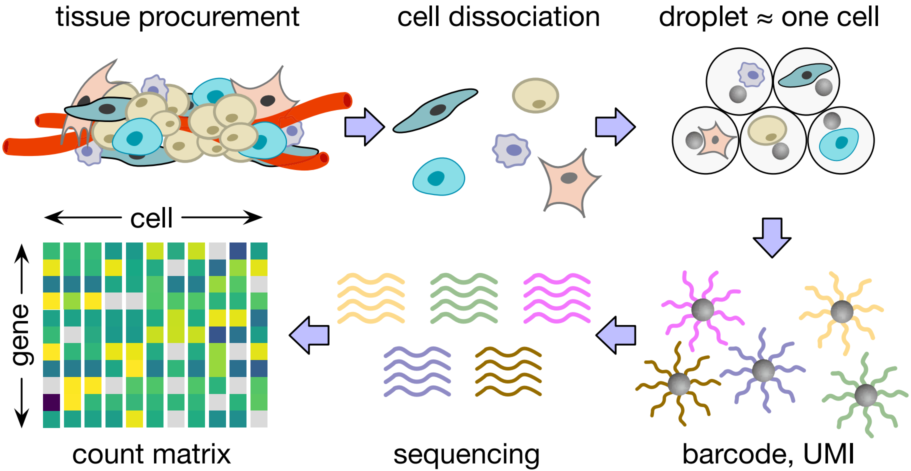


- Millions of the same **cell barcode** per bead
- 4^8 different **molecular barcodes** (UMIs) per bead

Technically, we can build up to a 65,536 × 16,777,216, gene × cell expression matrix in one single-cell RNA-seq experiment.

(Macosko et al. 2015)

Droplet-based single-cell sequencing pipeline

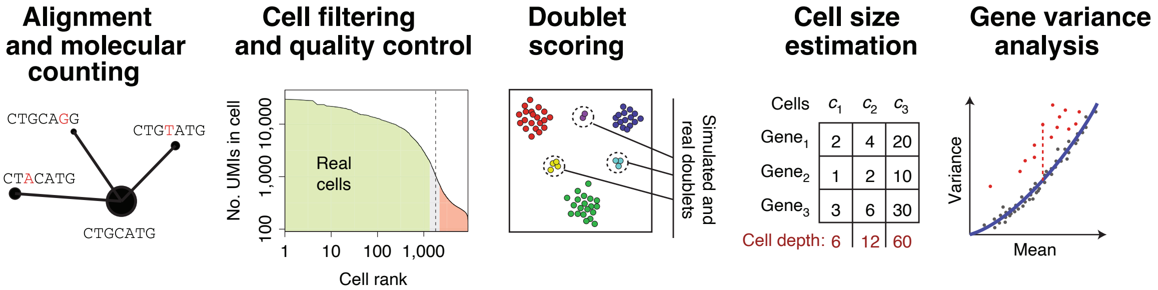


(Macosko et al. 2015)

Discussions: What are the potential challenges?

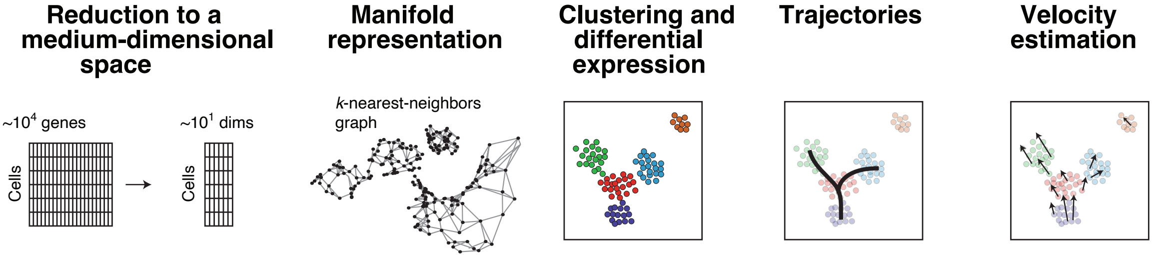
- What are the same challenges we had in bulk RNA-seq?
- What are the new challenges?

Overview of single-cell data analysis



(Kharchenko 2021)

Overview of single-cell data analysis cont'd



(Kharchenko 2021)

Today's lecture

1. Single-cell sequencing technology
2. Basic quality control
3. Data normalization across many batches

Example: Comprehensive immune profiling in pancreatic cancer

Multimodal mapping of the tumor and peripheral blood immune landscape in human pancreatic cancer

Approach:

- **Disease model:** Pancreatic ductal adenocarcinoma (PDA)
- **Technologies:** CyTOF, **single-cell RNA sequencing**, multiplex immunohistochemistry
- **Sample diversity:** Tumors, matched blood, and healthy tissues
- **Goal:** Comprehensive immune characterization

Key discoveries

- Revealed complex cellular immune networks
- Mapped patient-specific checkpoint receptor patterns
- Identified $CD8^+$ T cell states and transitions
- Discovered TIGIT as potential therapeutic target

(Steele et al., 2020)

Dataset overview: 41 samples

- 16 PDAC tissue
- 3 Adjacent normal tissue
- 16 PDAC PBMC
- 4 Healthy PBMC

Platform: 10x Genomics Chromium

`../data/GSE155698_PDAC_Steele2020/`

GEO: GSE155698

Sample directories

PDAC Tissue (16)

PDAC_TISSUE_1
PDAC_TISSUE_2
...
PDAC_TISSUE_16

PDAC PBMC (16)

PDAC_PBMC_1
PDAC_PBMC_2
...
PDAC_PBMC_16

Controls

AdjNorm_TISSUE_1
AdjNorm_TISSUE_2
AdjNorm_TISSUE_3

Healthy_PBMC_1
Healthy_PBMC_2
Healthy_PBMC_3
Healthy_PBMC_4

10x Genomics data format

```
PDAC_TISSUE_1/
|-- barcodes.tsv.gz    (9 KB)
|-- features.tsv.gz    (303 KB)
+-- matrix.mtx.gz      (17 MB)
```

- **`barcodes.tsv.gz`** - Cell IDs
- **`features.tsv.gz`** - Genes
- **`matrix.mtx.gz`** - Counts (sparse)

Matrix Market format

- Sparse matrix standard
- Memory efficient
- Compatible with Seurat, Scanpy

Why sparse? Most genes are not expressed in most cells

Loading 10x data with Seurat (R)

```r
library(Seurat)

data.dir <- "../data/GSE155698_PDAC_Steele2020/PDAC_TISSUE_1/"

pdac <- Read10X(data.dir = data.dir)
pdac <- CreateSeuratObject(counts = pdac,
                           project = "PDAC_TISSUE_1",
                           min.cells = 3,
                           min.features = 200)
pdac
```

```
## An object of class Seurat
## 19273 features across 1410 samples within 1 assay
## Active assay: RNA (19273 features, 0 variable features)
##  1 layer present: counts
```

Seurat object structure

- `SeuratObject` class stores everything
- Genes in rows, cells in columns
- Access counts: `GetAssayData(pdac)`
- Cell metadata: `pdac@meta.data`

```
head(pdac@meta.data, 3)
```

```
##                        orig.ident nCount_RNA nFeature_RNA
## AAACGAAAGTGGAAAG-1 PDAC_TISSUE_1        501          269
## AAACGAAGTAGGGTAC-1 PDAC_TISSUE_1      26196         4641
## AAACGAAGTCATAGTC-1 PDAC_TISSUE_1      72725         7990
```

```
head(rownames(pdac), 5)
```

```
## [1] "RP11-34P13.7"  "AL627309.1"    "AP006222.2"    "RP4-669L17.10"
## [5] "RP11-206L10.3"
```

Loading 10x data with scanpy (Python)

```python
import scanpy as sc

data_dir = "../data/GSE155698_PDAC_Steele2020/PDAC_TISSUE_1/"

adata = sc.read_10x_mtx(data_dir, var_names='gene_symbols', cache=True)
adata
```

```
## AnnData object with n_obs × n_vars = 1633 × 32738
##     var: 'gene_ids', 'feature_types'
```

AnnData object structure

- `AnnData` class stores everything
- Cells in rows, genes in columns (transposed from Seurat!)
- Access counts: `adata.X`
- Cell metadata: `adata.obs`, gene metadata: `adata.var`

```
print(adata.obs_names[:5].tolist())
```

```
## ['AAACGAAAGTGGAAAG-1', 'AAACGAAGTAGGGTAC-1', 'AAACGAAGTCATAGTC-1', 'AAACGCTGTAATCAGA-1', 'AAAGAACCATT
```

```
print(adata.var_names[:5].tolist())
```

```
## ['MIR1302-10', 'FAM138A', 'OR4F5', 'RP11-34P13.7', 'RP11-34P13.8']
```

Cell QC metrics with Seurat

```r
# Calculate mitochondrial gene percentage
pdac[["percent.mt"]] <- PercentageFeatureSet(pdac, pattern = "^MT-")

# View QC metrics
head(pdac@meta.data)
```

```
##                    orig.ident nCount_RNA nFeature_RNA percent.mt
## AAACGAAAGTGGAAAG-1 PDAC_TISSUE_1        501          269  12.574850
## AAACGAAGTAGGGTAC-1 PDAC_TISSUE_1      26196         4641  45.132845
## AAACGAAGTCATAGTC-1 PDAC_TISSUE_1      72725         7990  14.349948
## AAAGAACCATTAAAGG-1 PDAC_TISSUE_1      12734         4046   3.683053
## AAAGGATTCGGCTTGG-1 PDAC_TISSUE_1       5653         2200   3.290288
## AAAGGGCAGTAGCAAT-1 PDAC_TISSUE_1      20289         4772   5.865247
```

- `nCount_RNA` = total UMI counts per cell
- `nFeature_RNA` = number of genes detected
- `percent.mt` = % mitochondrial reads (high = dying cells)

Why high mitochondrial?

- MT genes: 13 protein-coding genes
- But 100s–1000s of mitochondria per cell
- MT RNA is abundant and stable

Why high mitochondrial?

- MT genes: 13 protein-coding genes
- But 100s–1000s of mitochondria per cell
- MT RNA is abundant and stable

Proportion effect:

Cytoplasmic RNA degrades $\rightarrow$ MT% goes up

Why does high MT implicate dying cells?

Conventional wisdom:

Membrane breaks $\rightarrow$ cytoplasmic RNA leaks $\rightarrow$ MT RNA stays

(Yates et al. 2025)

Why does high MT implicate dying cells?

Conventional wisdom:

Membrane breaks $\rightarrow$ cytoplasmic RNA leaks $\rightarrow$ MT RNA stays

But...

- Why wouldn't mitochondria also be damaged?
- Other organelles have membranes too (ER, nucleus)

(Yates et al. 2025)

Why does high MT implicate dying cells?

Conventional wisdom:

Membrane breaks → cytoplasmic RNA leaks → MT RNA stays

But...

- Why wouldn't mitochondria also be damaged?
- Other organelles have membranes too (ER, nucleus)

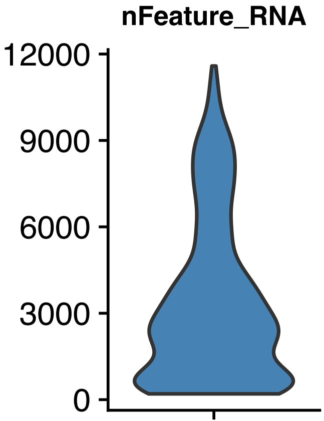
High MT% correlates with low quality — mechanism unclear

(Yates et al. 2025)

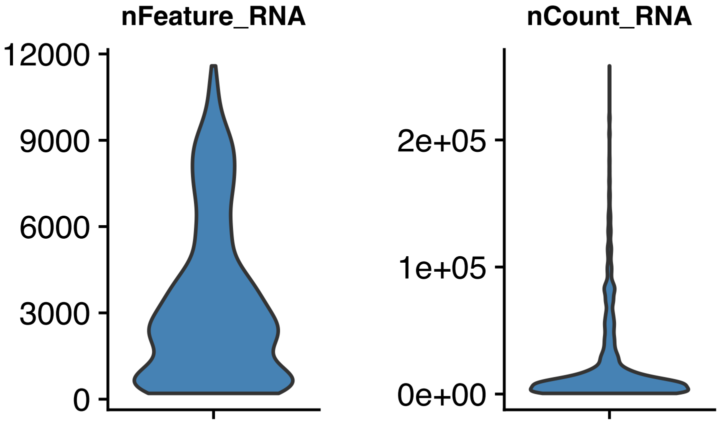
Seurat: QC visualization commands

```r
# Violin plots
VlnPlot(pdac, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"))

# Scatter plots
FeatureScatter(pdac, feature1 = "nCount_RNA", feature2 = "nFeature_RNA")
FeatureScatter(pdac, feature1 = "nCount_RNA", feature2 = "percent.mt")
```

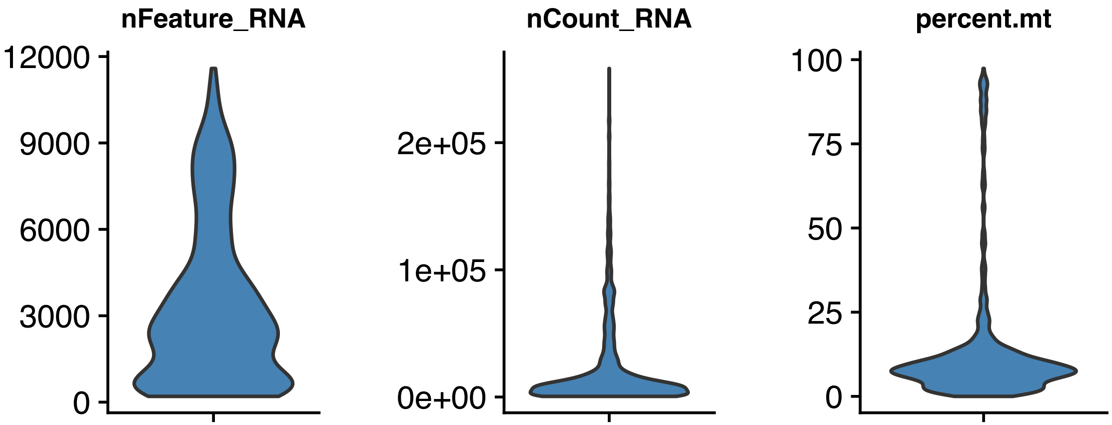
Visualizing QC metrics



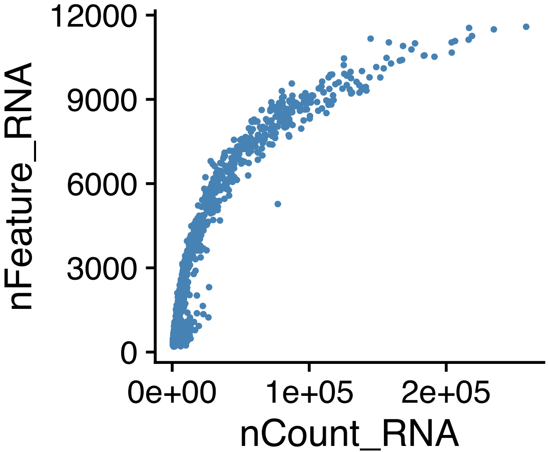
Visualizing QC metrics



Visualizing QC metrics

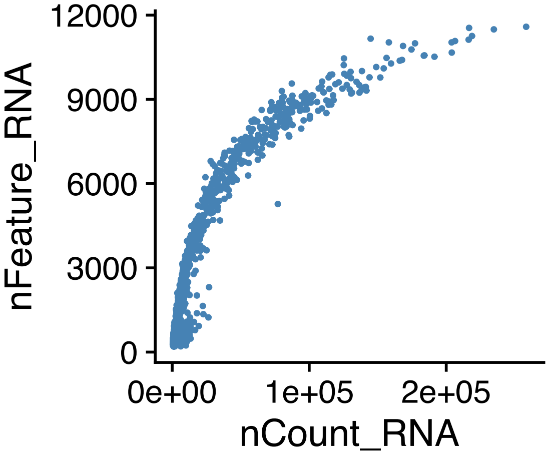


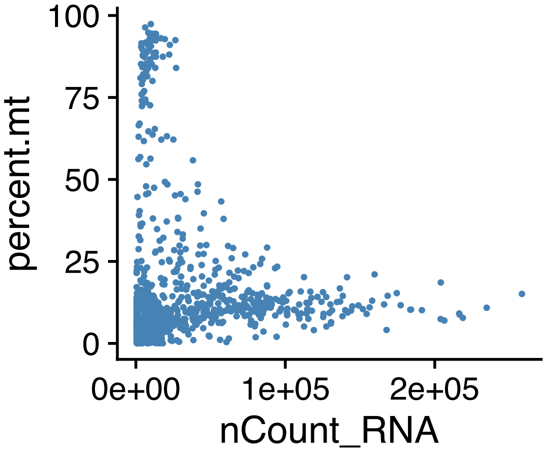
QC: Scatter plots



- Positive correlation expected
- Low-low → empty/dying

QC: Scatter plots





- Positive correlation expected
- Low-low → empty/dying

- High MT + low count → dying
- High MT + high count → keep?

Cell QC metrics with scanpy

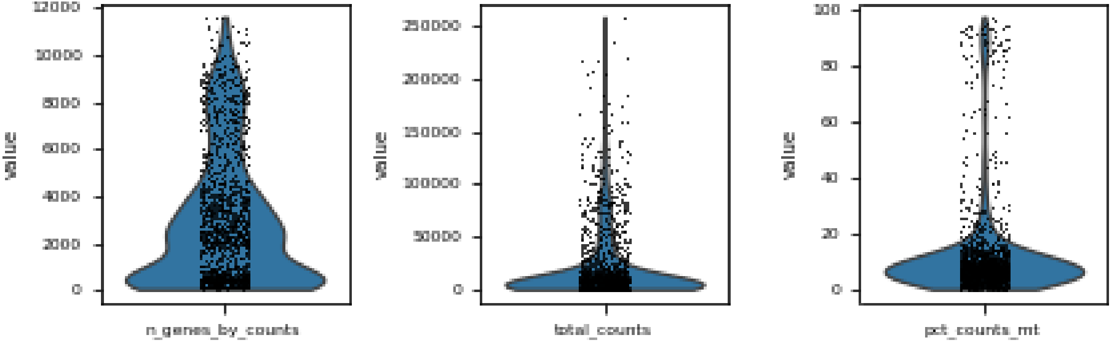
```
# Calculate QC metrics (mitochondrial genes start with "MT-")
adata.var['mt'] = adata.var_names.str.startswith('MT-')
sc.pp.calculate_qc_metrics(adata, qc_vars=['mt'], inplace=True)
adata.obs.head()[['n_genes_by_counts', 'total_counts', 'pct_counts_mt']]
```

```
##                     n_genes_by_counts  total_counts  pct_counts_mt
## AAACGAAAGTGGAAAG-1                269         501.0      12.574850
## AAACGAAGTAGGGTAC-1               4642       26197.0      45.131123
## AAACGAAGTCATAGTC-1               7995       72730.0      14.348962
## AAACGCTGTAATCAGA-1                158         835.0       4.431138
## AAAGAACCATTAAAGG-1               4047       12735.0       3.682764
```

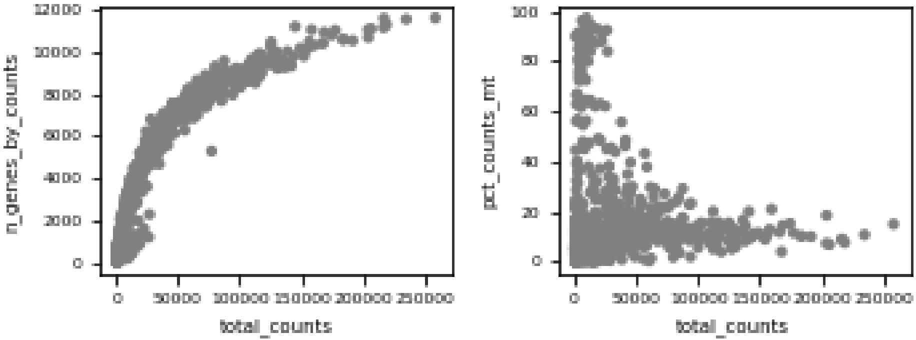
scanpy: QC visualization commands

```python
# Violin plots
sc.pl.violin(adata, ['n_genes_by_counts', 'total_counts', 'pct_counts_mt'])

# Scatter plots
sc.pl.scatter(adata, x='total_counts', y='n_genes_by_counts')
sc.pl.scatter(adata, x='total_counts', y='pct_counts_mt')
```

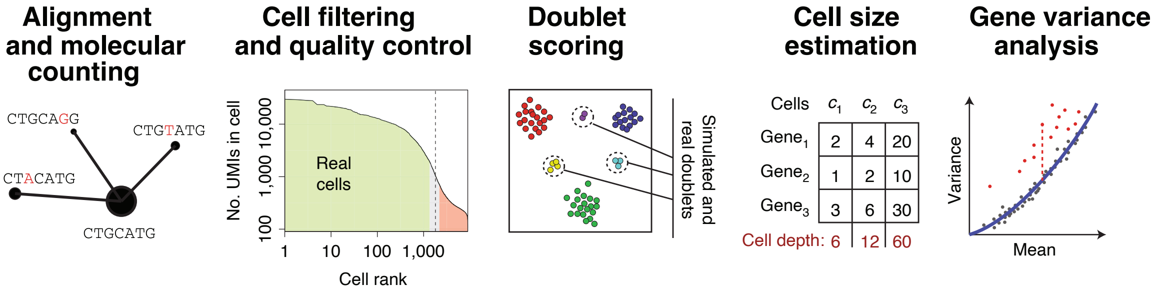
Visualizing QC metrics (scanpy)



QC: Scatter plots (scanpy)



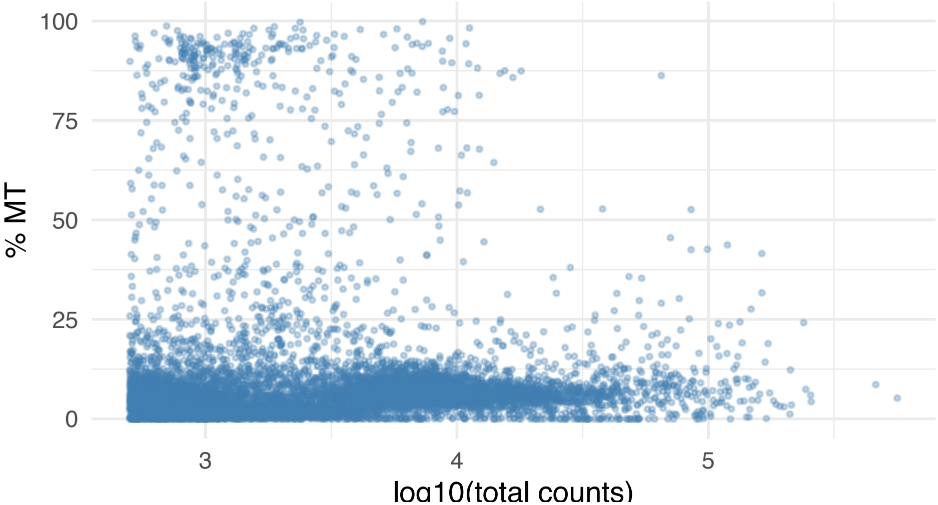
Overview: where are we?



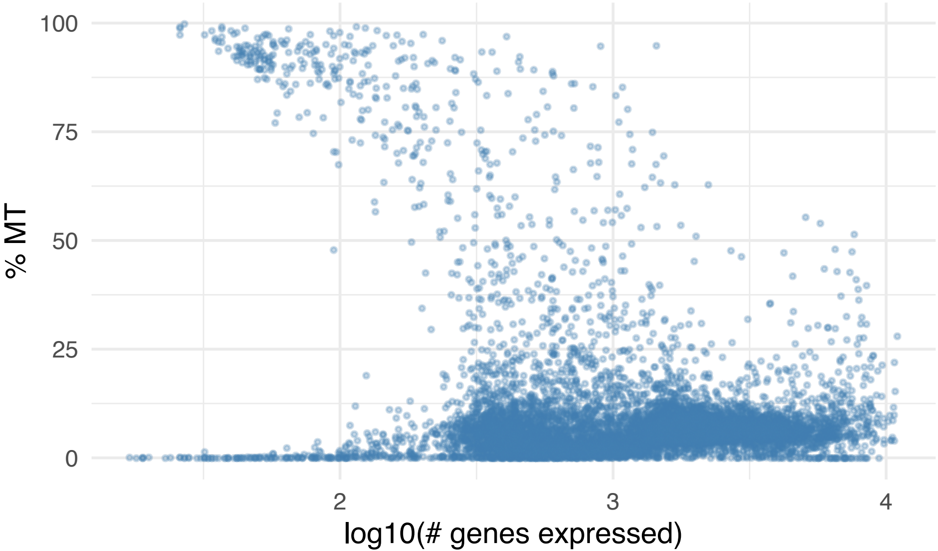
(Kharchenko 2021)

Cell stats: total counts vs. MT%

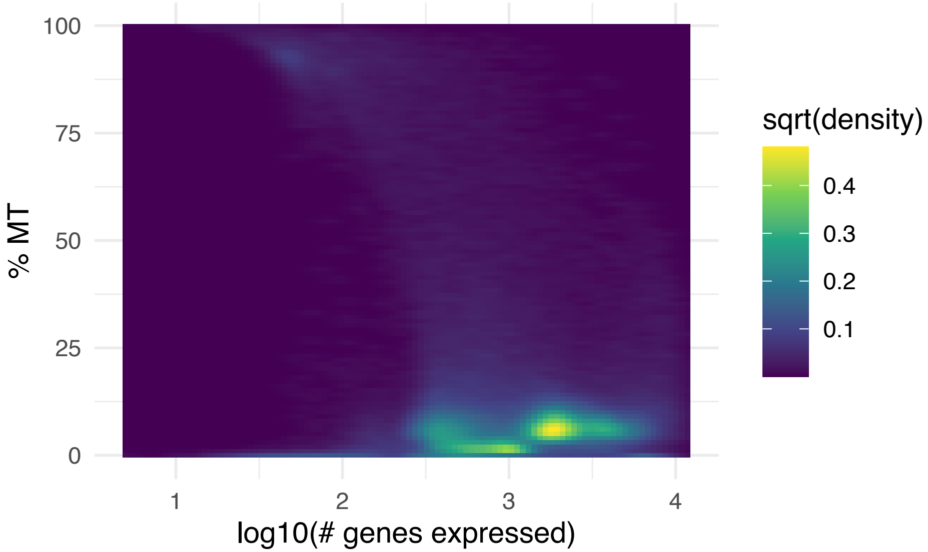
We have **136,386** cells across 38 samples.



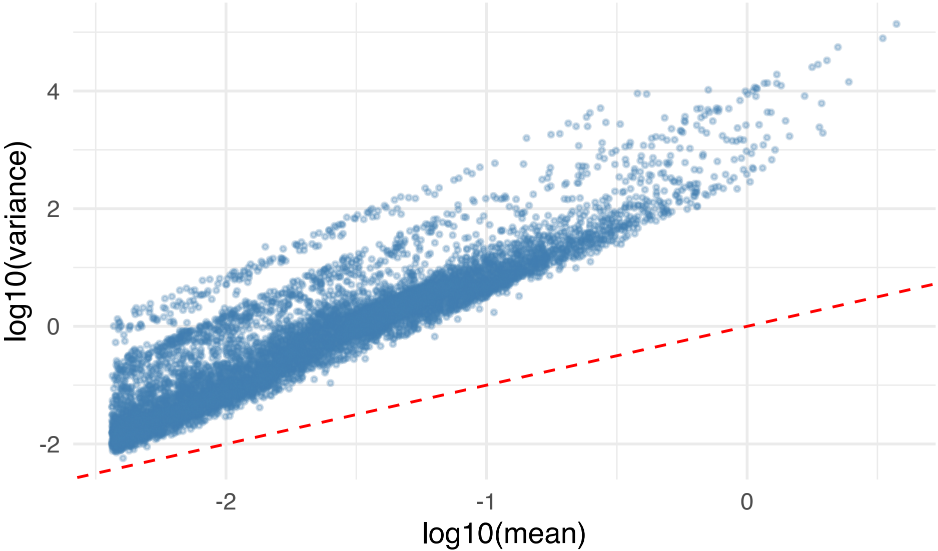
Cell stats: # genes expressed vs. MT%



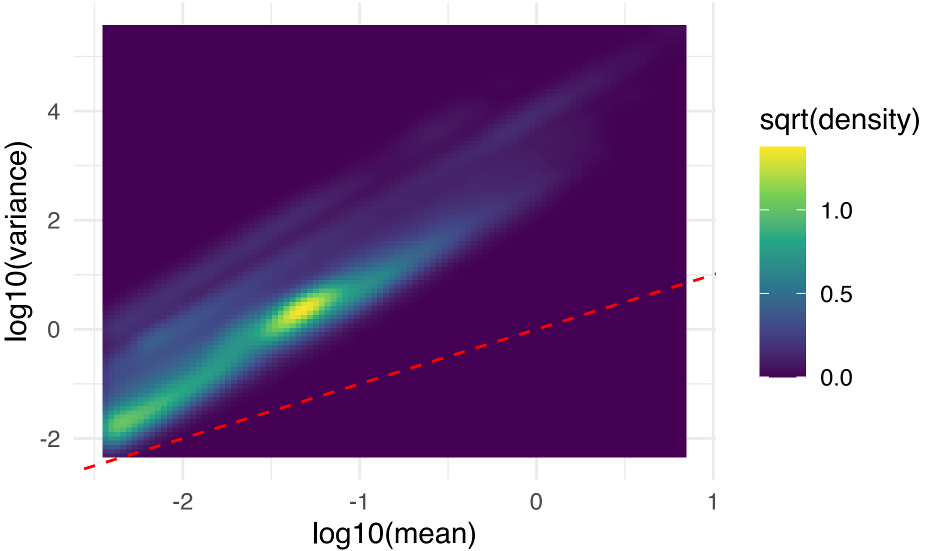
Cell stats: 2D density



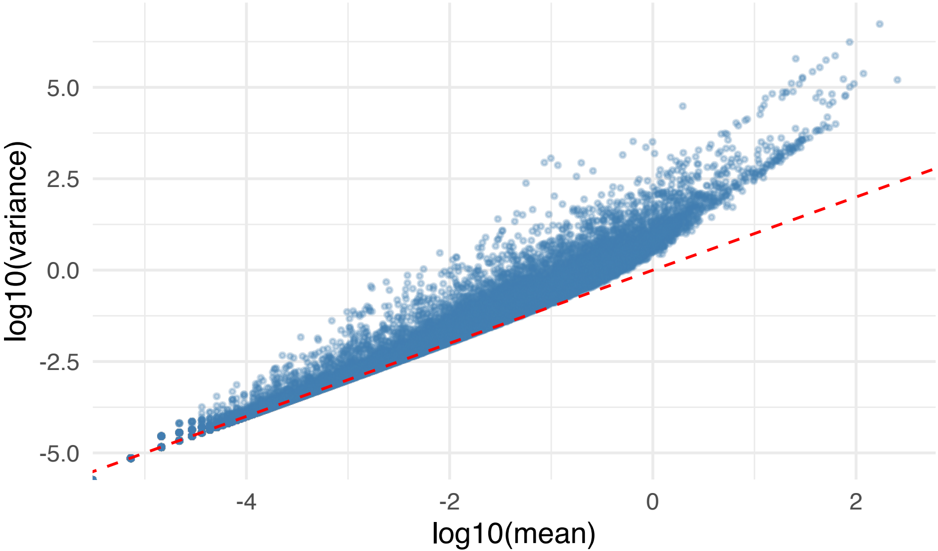
Mean-variance relationship: cells



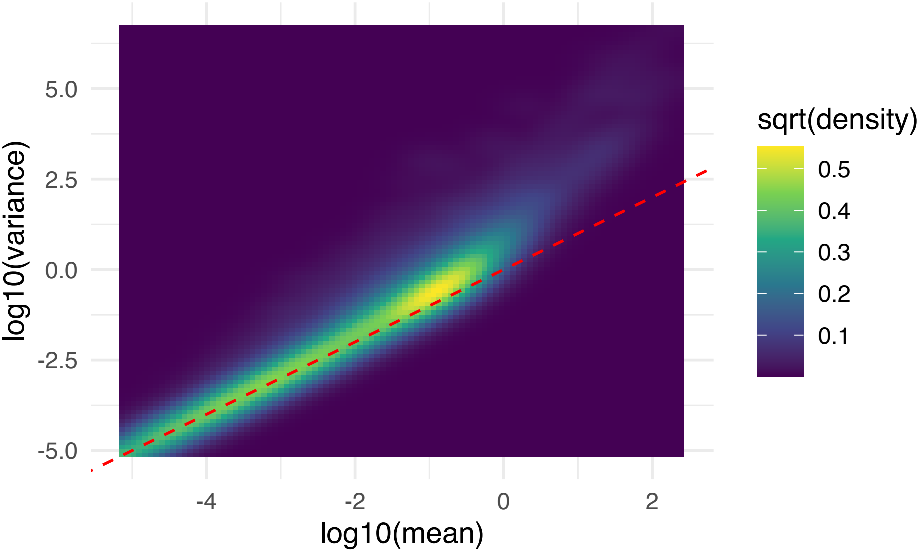
Mean-variance relationship: cells (density)



Mean-variance relationship: genes



Mean-variance relationship: genes (density)



Can we use the count data as they are?

- In bulk RNA-seq, we transformed the count data to adjust size factors that vary across samples.
- In single-cell data analysis, we generally care a lot about zero values, not knowing whether they come from actual "no expression" or technical drop-out.
- Should we take some sort of `voom` transformation?

Transformation methods for scRNA-seq

Raw counts

$$Y$$

Delta method

$$\log(Y/s + 1)$$

GLM residual

$$\frac{Y - \mu}{\sqrt{\mu + \alpha\mu^2}}$$

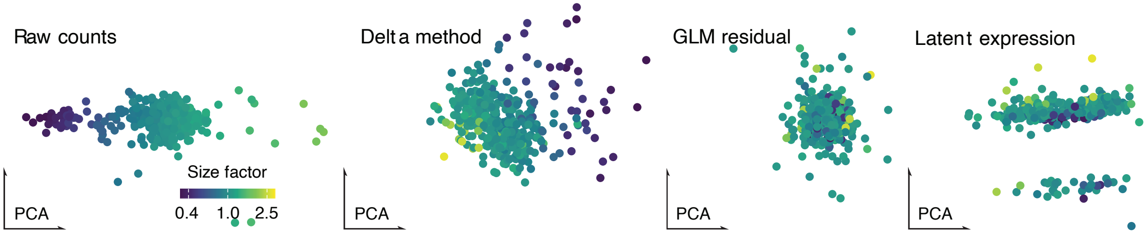
Latent expression

$$Y \sim \text{Poisson}(M)$$

$$M \sim \text{logNormal}(\mu,$$

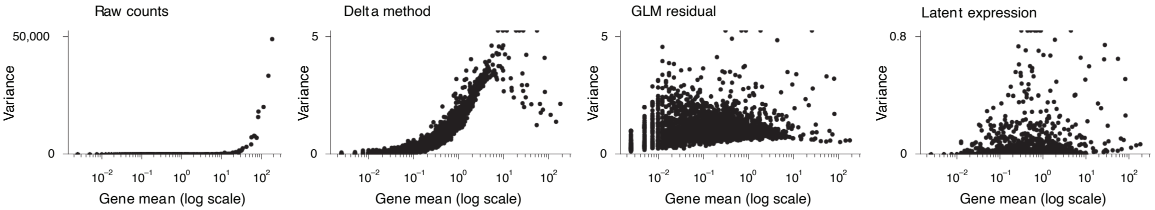
(Ahlmann-Eltze and Huber 2023)

Effect of transformation on PCA



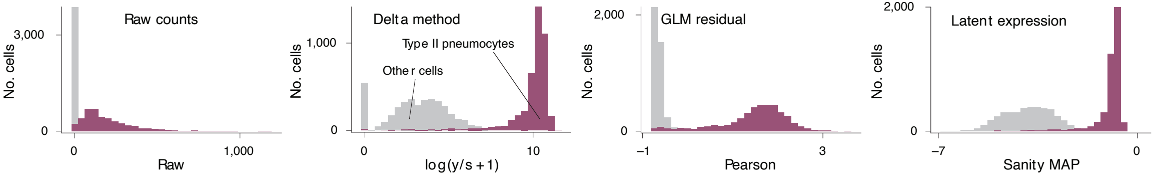
(Ahlmann-Eltze and Huber 2023)

Mean-variance relationship after transformation



(Ahlmann-Eltze and Huber 2023)

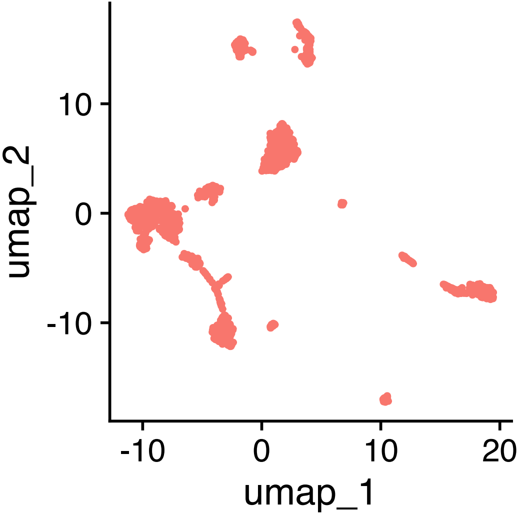
Gene expression distribution after transformation



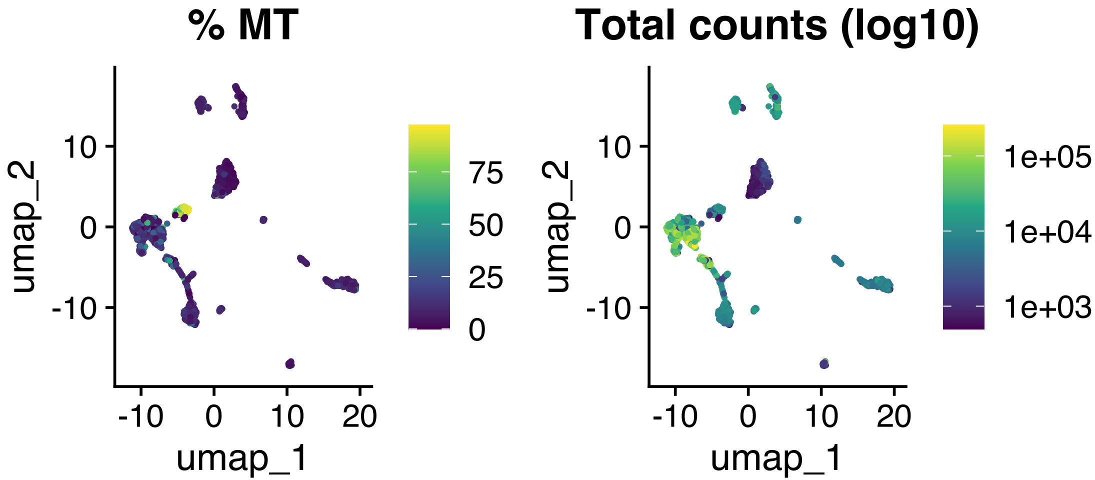
(Ahlmann-Eltze and Huber 2023)

What would happen after cell Q/C?

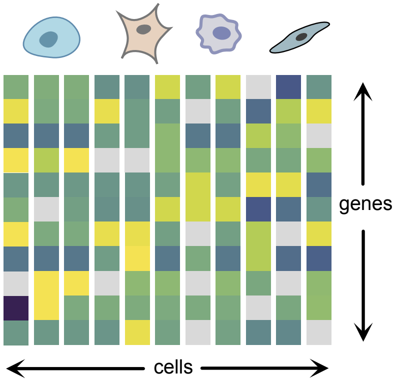
```r
pdac <- pdac |>
    NormalizeData() |>
    FindVariableFeatures() |>
    ScaleData() |>
    RunPCA() |>
    RunUMAP(dims = 1:20)

DimPlot(pdac,
        reduction = "umap")
```



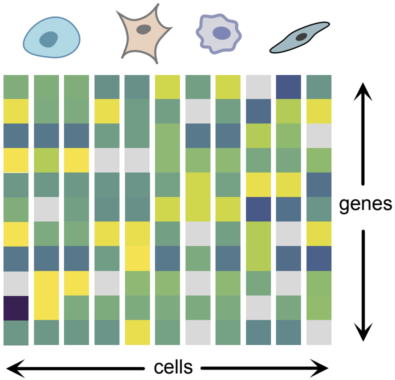
What would happen after Q/C? Let's see..

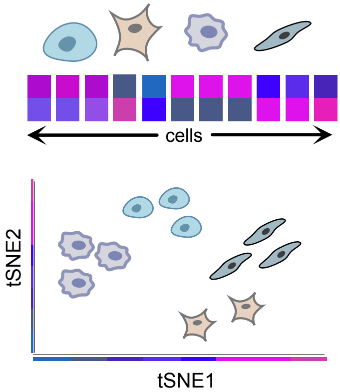


It's useful to show high-dimensional data in 2D



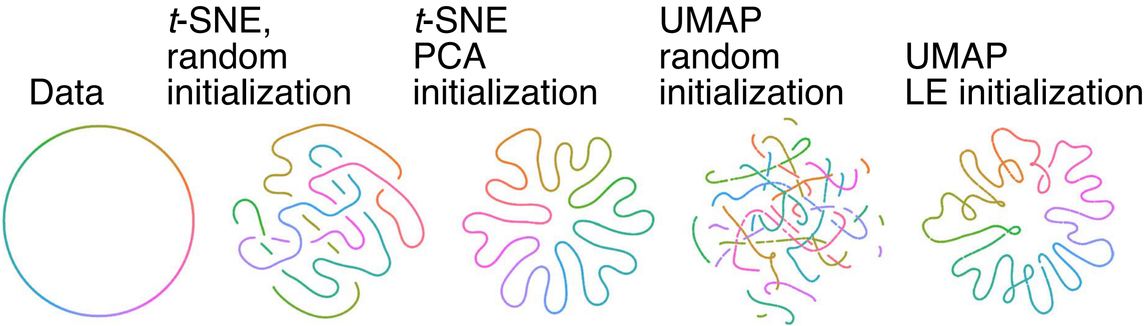
It's useful to show high-dimensional data in 2D





`tSNE`: t-distributed Stochastic Neighbourhood Embedding (Van der Maaten & Hinton, 2008).

Warning: Don't over interpret 2D embedding



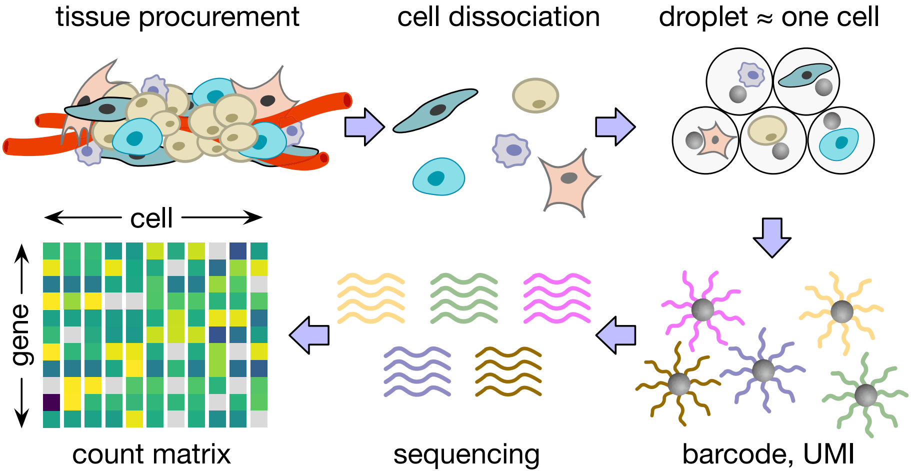
(Kobak and Linderman 2021)

Don't over-interpret 2D embedding results

Check out these papers:

- Initialization is critical for preserving global data structure in both t-SNE and UMAP
- The art of using t-SNE for single-cell transcriptomics
- Dimensionality reduction for visualizing single-cell data using UMAP

What if we capture more than one cell in a droplet?



(Macosko et al. 2015)

What is a doublet in single-cell data?

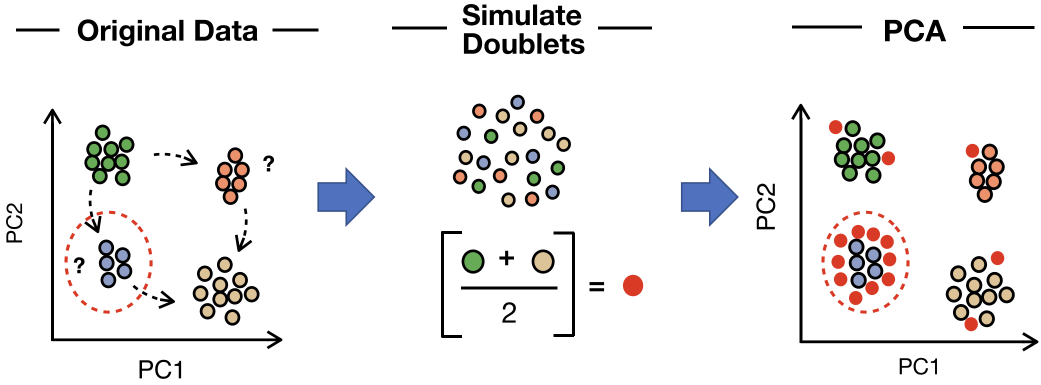
Biological/technical definition:

- One or more cells captured (usually at most two cells by chance)
- Thus, multiple cells accidentally share the same cell barcode sequence
- Not so clear in general... since we missed the chance to assign different tags to different cells encapsulated in the same droplet.

Statistical definition:

- If marker genes of multiple cell types are simultaneously expressed...
- An unvetted approach: Find ambiguous/intermediate coordinates in PCA/tSNE/UMAP (after removing ambient cells).

Idea: k-Nearest Neighbour classification for doublet detection



McGinnis et al. Cell Systems (2019)

Can we create artificial doublets?

A straightforward definition (used in `DoubletFinder`):

For each cell i:

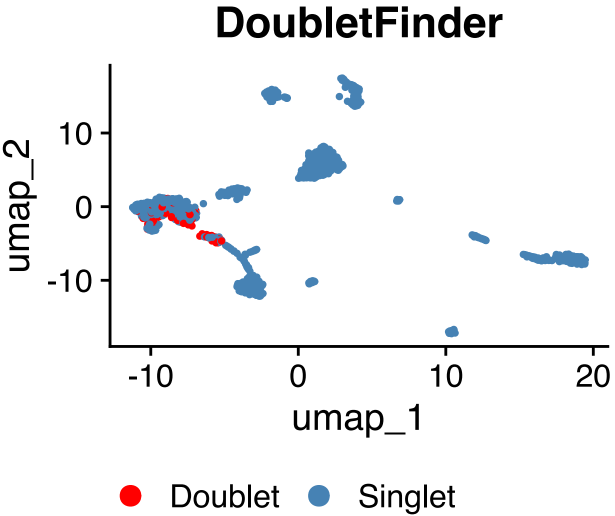
- Take some other j by random selection
- Create an artificial doublet
$$\tilde{\mathbf{x}} \leftarrow \frac{1}{2}(\mathbf{x}_i + \mathbf{x}_j)$$

Some thought questions:

- Doublets within the same cell type?
- Doublets between the different cell types?

DoubletFinder on PDAC_TISSUE_1

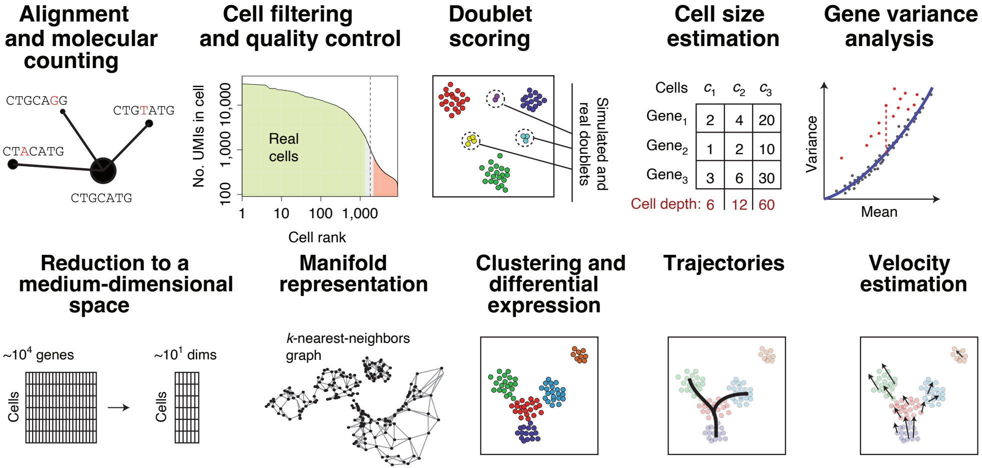
```r
library(DoubletFinder)

sweep.res <- paramSweep(
    pdac, PCs = 1:20)
sweep.stats <- summarizeSweep(
    sweep.res, GT = FALSE)
bcmvn <- find.pK(sweep.stats)

pdac <- doubletFinder(
    pdac, PCs = 1:20,
    pN = 0.25, pK = pK,
    nExp = nExp_poi)
```



Today's lecture

1. Single-cell sequencing technology
2. Basic quality control
3. Data normalization across many batches

Where are we?

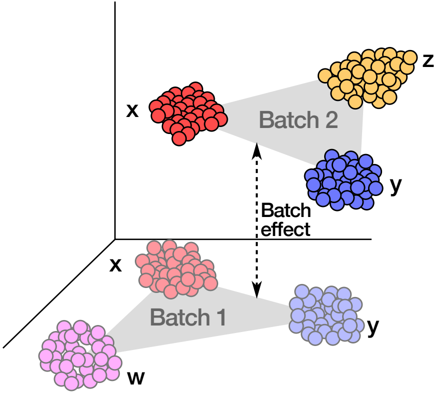


(Kharchenko 2021)

Why batch normalization?

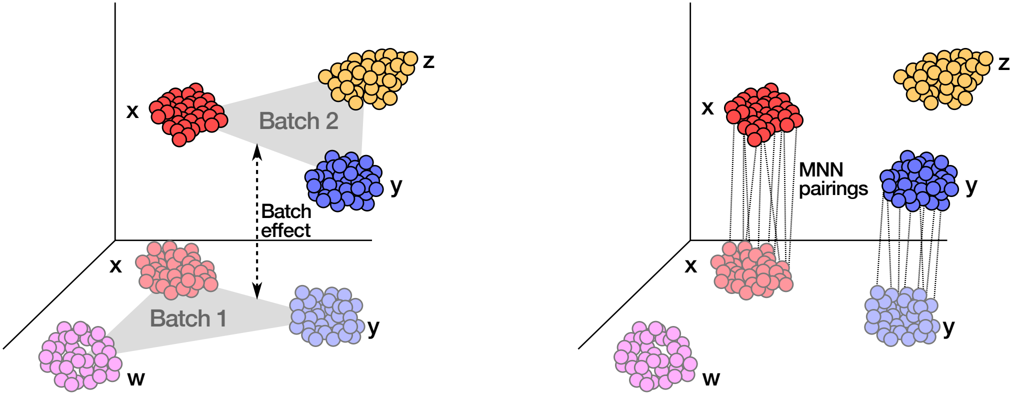
- Single-cell experiments are often performed across **multiple samples, batches, or technologies**
- Technical variation can cause cells to cluster by **batch** rather than **cell type**
- Without correction, biological signals are confounded with technical artifacts
- **Goal**: Remove technical variation while preserving biological heterogeneity

Batch normalization aims to minimize the difference between nearest cells across different batches



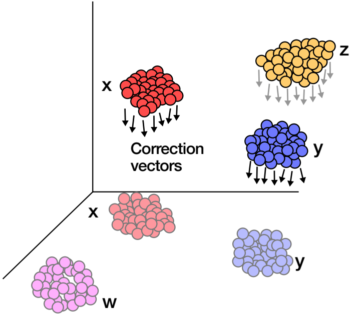
(Haghverdi et al. 2016)

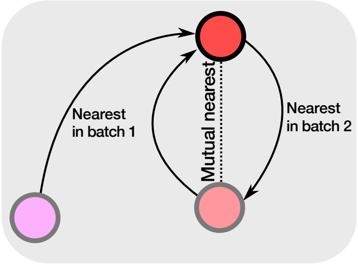
Batch normalization aims to minimize the difference between nearest cells across different batches



(Haghverdi et al. 2016)

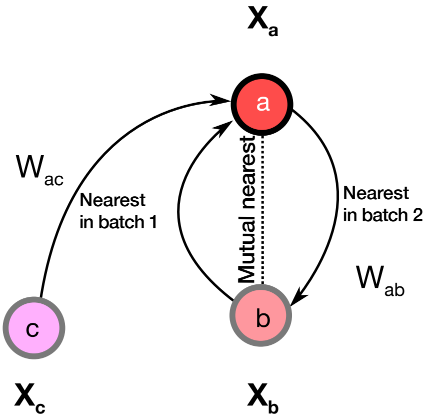
Batch normalization aims to minimize the difference between nearest cells across different batches





(Haghverdi et al. 2016)

Batch normalization aims to minimize the difference between nearest cells across different batches



What is the gap Δ between the batches?

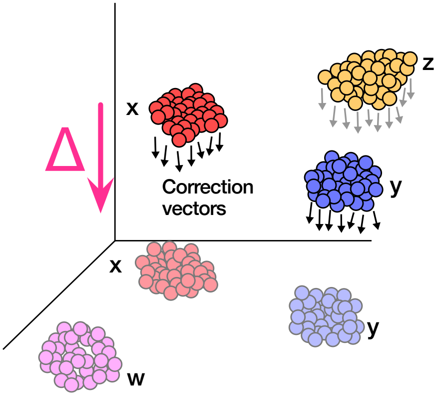
$$\min_{\Delta} \sum_{a,b} W_{ab} \|\mathbf{x}_a - \mathbf{x}_b - \Delta\|_2$$

A key assumption:

$$0 \approx W_{ac} < W_{ab}$$

(Haghverdi et al., 2016)

Batch normalization aims to minimize the difference between nearest cells across different batches



What is the gap Δ between the batches?

$$\min_{\Delta} \sum_{a,b} W_{ab} \|\mathbf{x}_a - \mathbf{x}_b - \Delta\|_2$$

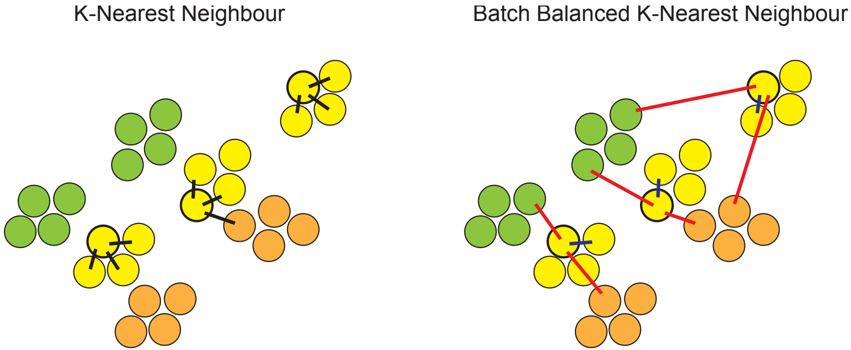
Fixed point (local) optimal solution:

$$\Delta \leftarrow \frac{\sum_{a,b} W_{ab}(\mathbf{x}_a - \mathbf{x}_b)}{\sum_b W_{ab}}$$

(Haghverdi et al. 2016)

A batch-balancing k-nearest neighbour graph

BBKNN method strikes balance between over- and under-normalization

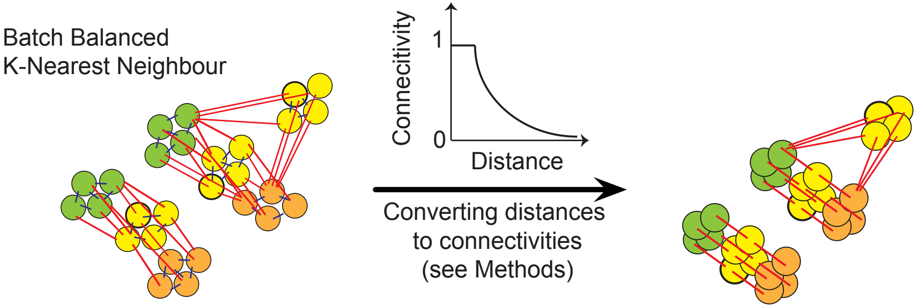


What kind of differences are due to inter-batch, technical discrepancy, not inter-cell-type divergence?

(Polański et al. 2019)

A batch-balancing k-nearest neighbour graph

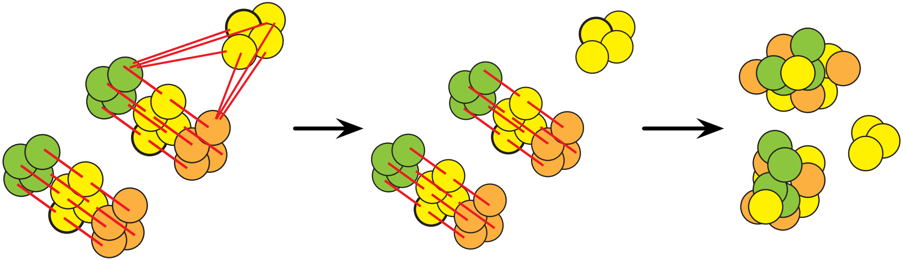
BBKNN method strikes balance between over- and under-normalization



(Polański et al. 2019)

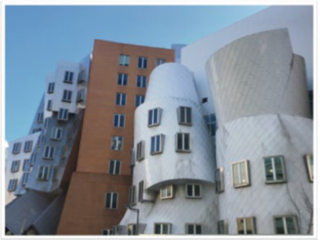
A batch-balancing k-nearest neighbour graph

BBKNN method strikes balance between over- and under-normalization



(Polański et al. 2019)

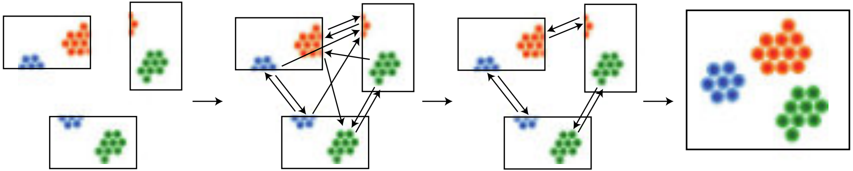
snapshots
of many data



panorama stitched
together

How do we integrate multiple samples/batches?

Scanorama: mutual nearest neighbourhood-based data integration



Collect many single-cell RNA-seq experiments

Find nearest neighbours across data sets

Keep mutually neighbouring cell pairs

Create single-cell panorama

(Hie et al. 2019)

Canonical Correlation Analysis (CCA)

- CCA finds linear combinations of features that are **maximally correlated** between two datasets
- Given two data matrices X_1 and X_2, find vectors a and b such that:

$$\mathrm{corr}(X_1 a, X_2 b) \text{ is maximized}$$

- The projected values $X_1 a$ and $X_2 b$ are called **canonical variates**
- Multiple canonical variates can be found (orthogonal to previous ones)

CCA for batch integration

Intuition: Cells of the same type should have similar gene expression patterns, regardless of batch

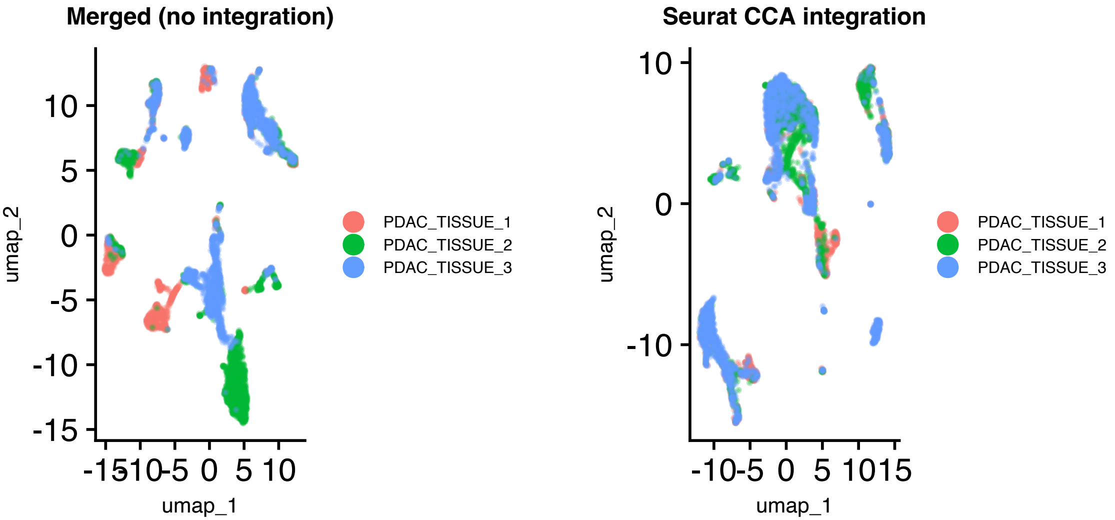
CCA for batch integration

Intuition: Cells of the same type should have similar gene expression patterns, regardless of batch

Seurat CCA integration:

1. Run CCA on each pair of datasets to find shared correlation structure
2. Identify **anchor cells** — pairs of cells (one from each batch) that are mutual nearest neighbours in CCA space
3. Use anchors to compute correction vectors that align the datasets

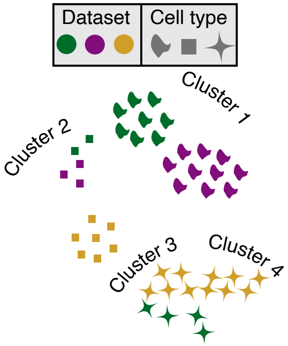
Batch normalization with Seurat



Seurat integration workflow

```r
# 1. Preprocess each sample separately
seu_list <- lapply(seu_list, function(seu) {
    seu |> NormalizeData() |> FindVariableFeatures()
})

# 2. Find integration anchors
features <- SelectIntegrationFeatures(seu_list)
anchors <- FindIntegrationAnchors(seu_list, anchor.features = features)

# 3. Integrate data
seu_integrated <- IntegrateData(anchors)

# 4. Standard workflow on integrated data
seu_integrated <- seu_integrated |>
    ScaleData() |> RunPCA() |> RunUMAP(dims = 1:20)
```

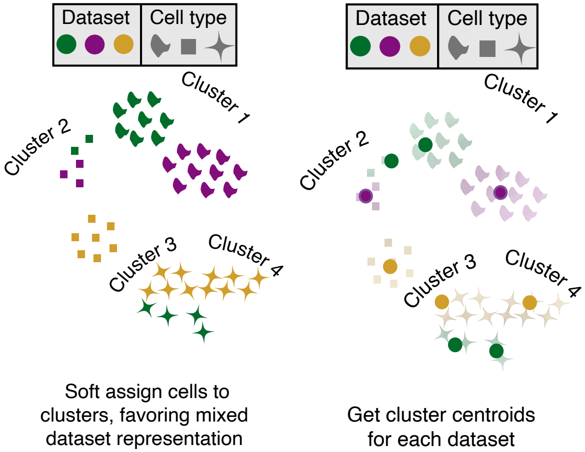
Harmony algorithm



Soft assign cells to clusters, favoring mixed dataset representation

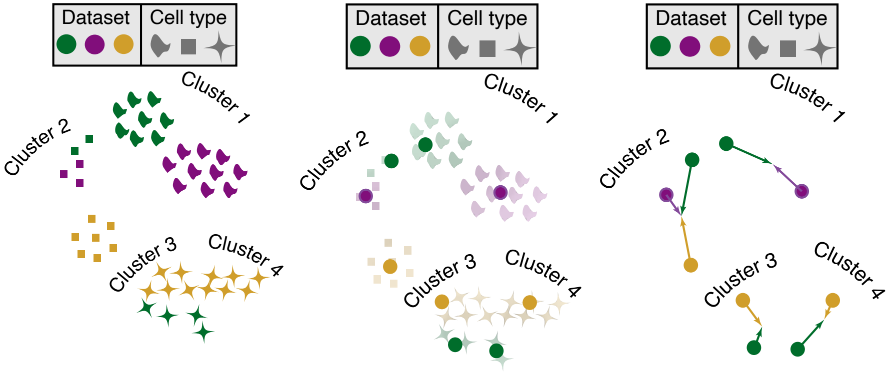
(Korsunsky et al. 2019)

Harmony algorithm



(Korsunsky et al. 2019)

Harmony algorithm



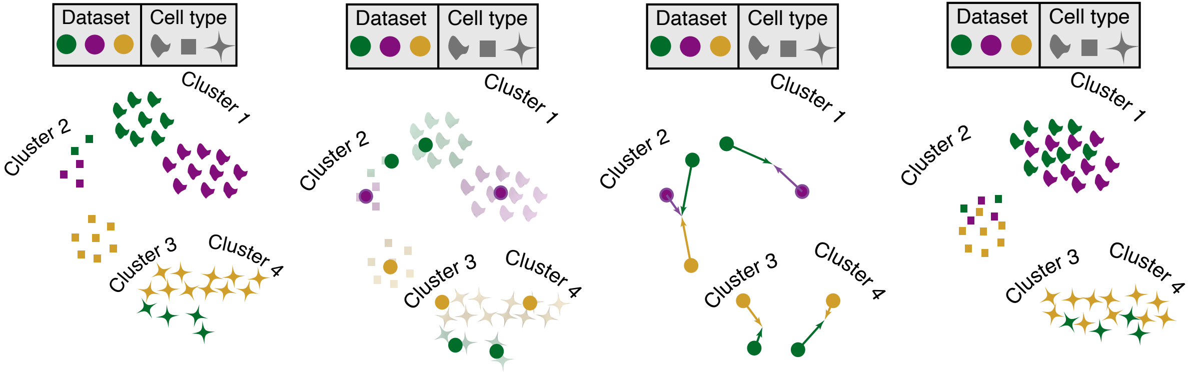
Soft assign cells to clusters, favoring mixed dataset representation

Get cluster centroids for each dataset

Get dataset correction factors for each cluster

(Korsunsky et al. 2019)

Harmony algorithm



Soft assign cells to clusters, favoring mixed dataset representation

Get cluster centroids for each dataset

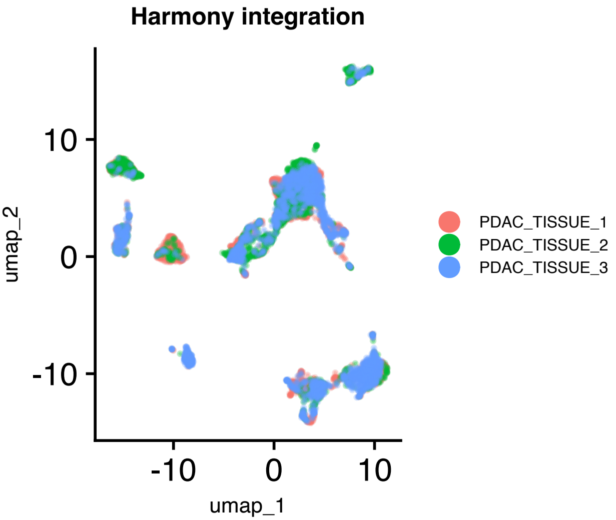
Get dataset correction factors for each cluster

Move cells based on soft cluster membership

(Korsunsky et al. 2019)

Harmony: a fast alternative

```r
library(harmony)

seu_harmony <- seu_merged |>
    RunHarmony("orig.ident") |>
    RunUMAP(reduction = "harmony",
            dims = 1:20)
```



Reference

Ahlmann-Eltze, Constantin, and Wolfgang Huber. 2023. "Comparison of Transformations for Single-Cell RNA-Seq Data." *Nat. Methods*, April, 1–8.

Haghverdi, Laleh, Maren Büttner, F Alexander Wolf, Florian Buettner, and Fabian J Theis. 2016. "Diffusion Pseudotime Robustly Reconstructs Lineage Branching." *Nat. Methods* 13 (10): 845–48.

Hie, Brian, Bryan Bryson, and Bonnie Berger. 2019. "Efficient Integration of Heterogeneous Single-Cell Transcriptomes Using Scanorama." *Nat. Biotechnol.* 37 (6): 685–91.

Reference

Kharchenko, Peter V. 2021. "The Triumphs and Limitations of Computational Methods for scRNA-Seq." *Nat. Methods*, June, 1–10.

Kobak, Dmitry, and George C Linderman. 2021. "Initialization Is Critical for Preserving Global Data Structure in Both t -SNE and UMAP." *Nat. Biotechnol.*, February, 1–2.

Korsunsky, Ilya, Nghia Millard, Jean Fan, et al. 2019. "Fast, Sensitive and Accurate Integration of Single-Cell Data with Harmony." *Nat. Methods*, November.

Macosko, Evan Z, Anindita Basu, Rahul Satija, et al. 2015. "Highly Parallel Genome-Wide Expression Profiling of Individual Cells Using Nanoliter Droplets." *Cell* 161 (5): 1202–14.

Reference

Polański, Krzysztof, Jong-Eun Park, Matthew D Young, Zhichao Miao, Kerstin B Meyer, and Sarah A Teichmann. 2019. "BBKNN: Fast Batch Alignment of Single Cell Transcriptomes." *Bioinformatics*, August.

Steele, Nina G, Eileen S Carpenter, Samantha B Kemp, et al. 2020. "Multimodal Mapping of the Tumor and Peripheral Blood Immune Landscape in Human Pancreatic Cancer." *Nature Cancer* 1 (11): 1097–112.

Yates, Josephine, Agnieszka Kraft, and Valentina Boeva. 2025. "Filtering Cells with High Mitochondrial Content Depletes Viable Metabolically Altered Malignant Cell Populations in Cancer Single-Cell Studies." *Genome Biol.* 26 (1): 91.